

System Design Final Project Report

Author: Sujal Pradeep Warke

Roll No: 150096724012

Cohort: Mark Zuckerberg (2024-2028)

Github Repo Link: https://github.com/sujalwarke28/systemDesign_VideoStreaming.git

1. Problem Statement

The objective was to design and implement a scalable, highly available video streaming platform demonstrating system design principles. The platform must handle large file uploads, serve content via HTTP Range Requests for seeking, and support social features.

2. Proposed Solution

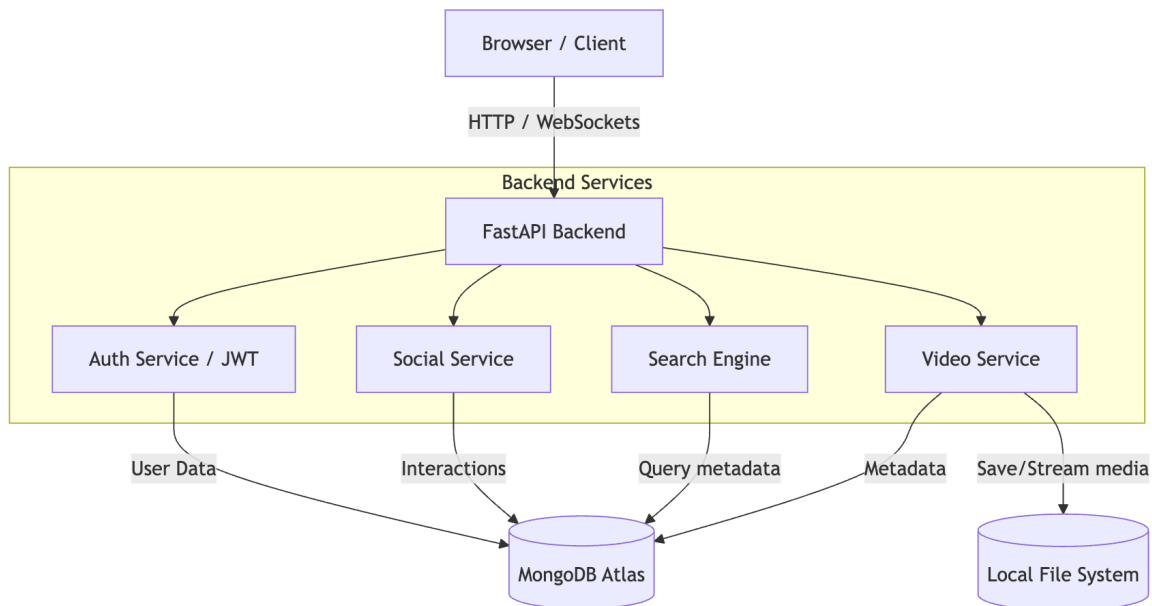
A microservice-ready architecture was adopted, featuring a FastAPI backend for asynchronous I/O performance. MongoDB Atlas was chosen for flexible metadata management. Local storage acts as a placeholder for a future cloud blob store (like S3).

3. Architecture

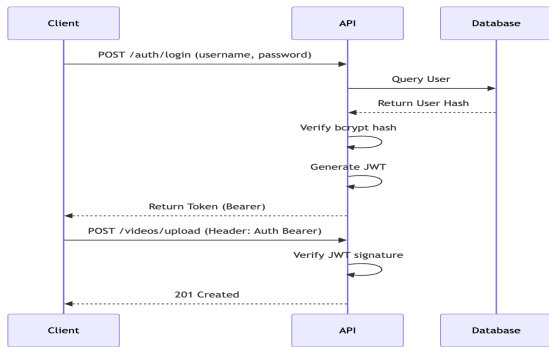
High-Level Architecture (3-Tier Model)

1. **Presentation Layer (Frontend):** Native HTML5, Bootstrap 5, and Vanilla JavaScript rendered dynamically using FastAPI's Jinja2 templating engine.
2. **Application Layer (Backend):** Python FastAPI handles all business logic, routing, authentication, and database communications asynchronously.
3. **Data Layer (Database & Storage):** MongoDB Atlas (NoSQL) is used for flexible document storage (users, videos, likes, comments). The local server file system acts as the media storage volume.

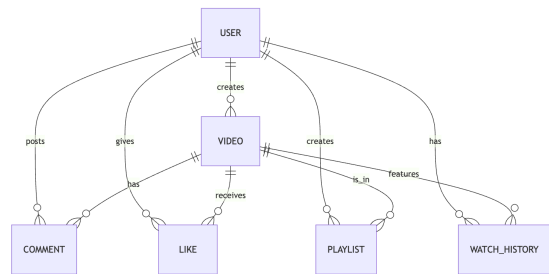
Component Architecture Diagram



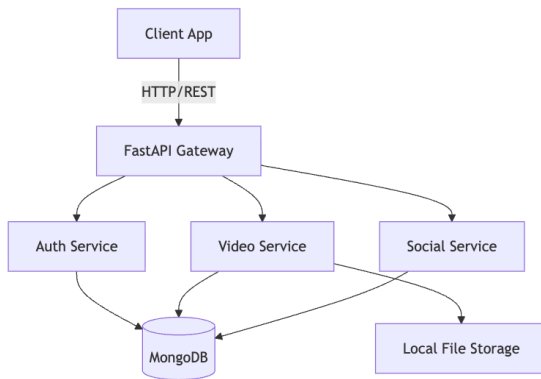
Authentication Flow



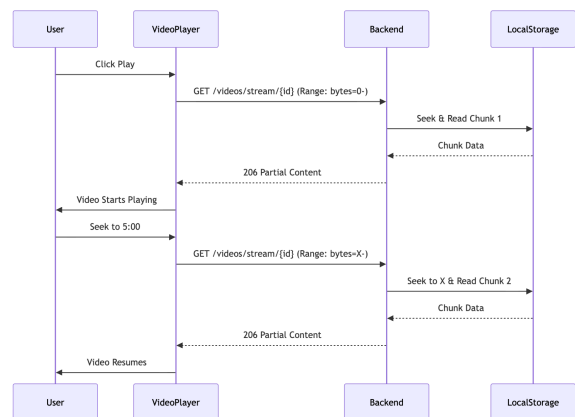
Database ER Diagram



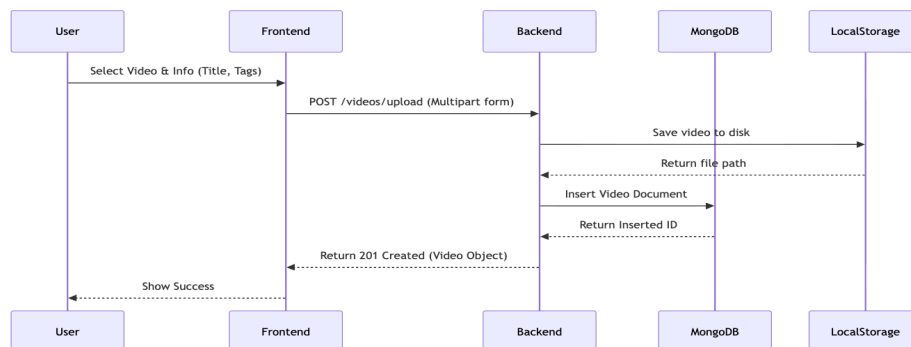
System Architecture



Video Streaming Sequence

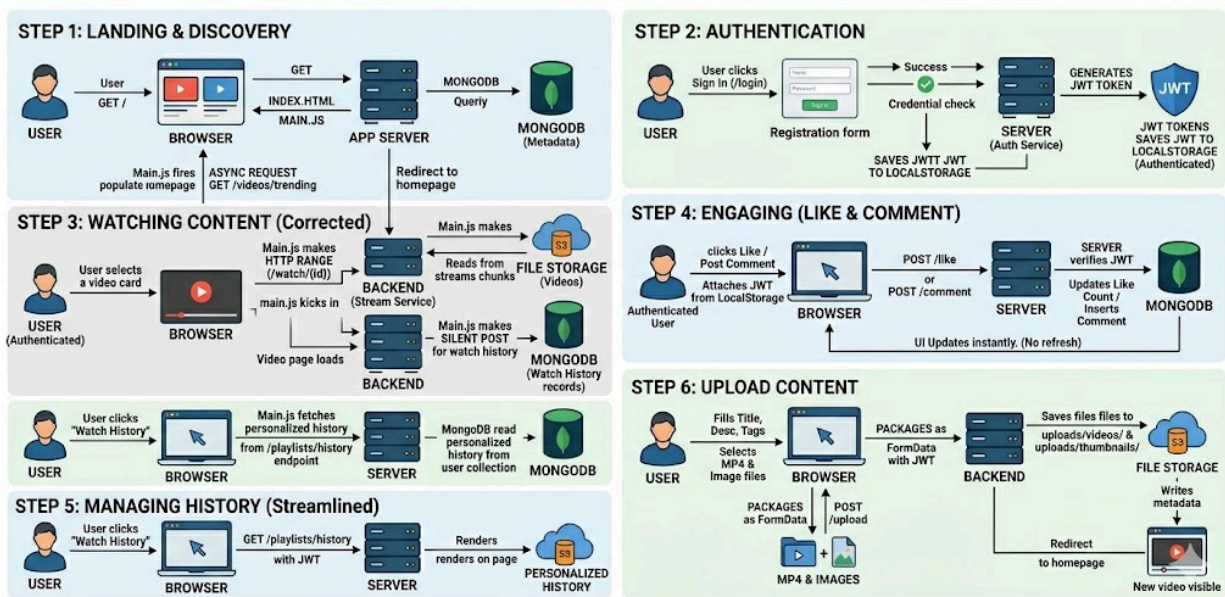


Video Upload Sequence



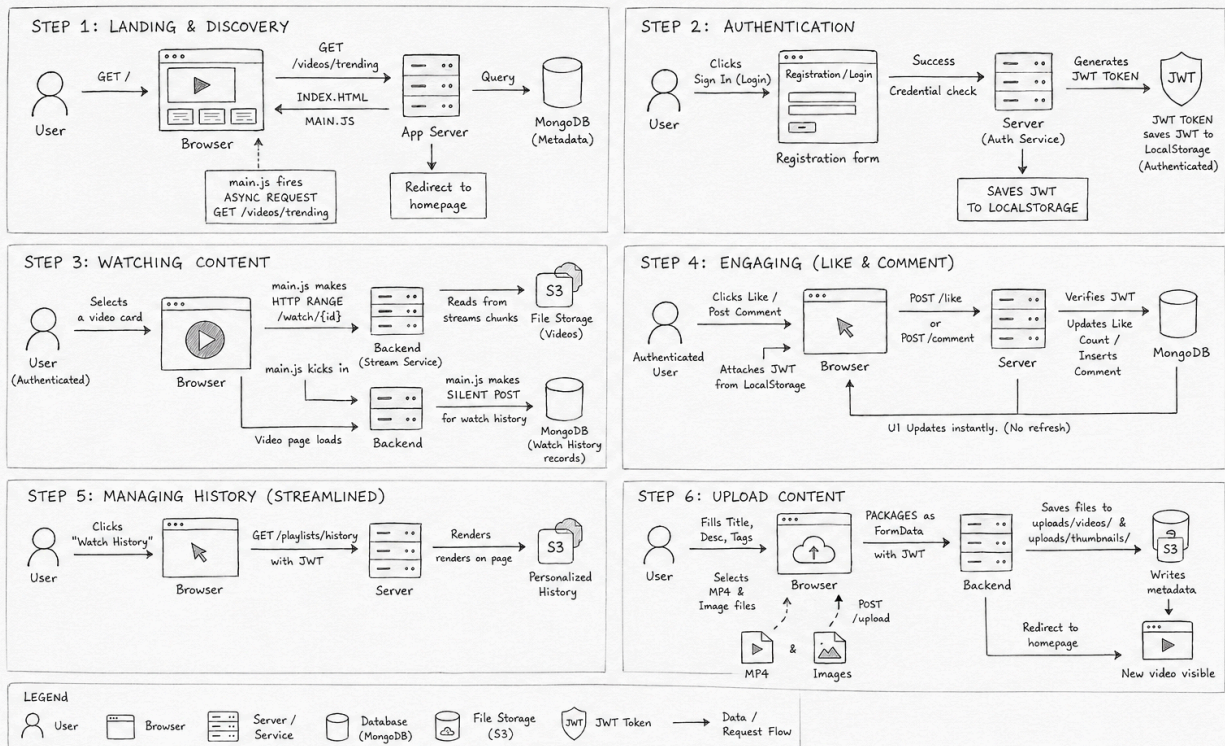
High Fidelity Diagram

STEP-BY-STEP USER JOURNEY: Landing → Auth → Watch → Engage → History → Upload



Low Fidelity Diagram

STEP-BY-STEP USER JOURNEY: Landing → Auth → Watch → Engage → History → Upload



- **Presentation Layer:** Native HTML / JS / Bootstrap 5 (Jinja2 UI)
- **Application Layer:** FastAPI API Gateway & Service Routes
- **Data Layer:** MongoDB (Users, Videos, Interactions)
- **Storage Layer:** Local volume for `uploads/videos/`

Detailed Technology Stack

1. Frontend Technologies

- **HTML5 & CSS3:** Core markup and styling.
- **Bootstrap 5:** Responsive grid framework and pre-built UI components (Modals, Dropdowns).
- **Vanilla JavaScript:** Handles asynchronous fetch requests for liking, commenting, searching, and uploading without requiring page reloads.
- **Jinja2:** Server-side template rendering for injecting dynamic variables (like the page title and API endpoints) directly into the HTML before serving it to the client.

2. Backend Technologies

- **FastAPI:** A modern, fast (high-performance) web framework for building APIs with Python 3.10+ based on standard Python type hints.
- **Uvicorn:** ASGI web server implementation for Python.
- **PyJWT:** For generating and verifying JSON Web Tokens (JWT) used in stateless authentication.
- **Passlib & bcrypt:** For secure password hashing.

3. Database and Storage Technologies

- **MongoDB Atlas:** Cloud-hosted NoSQL database. Documents are stored in BSON format, making it highly flexible for dynamic schemas like Video metadata.
- **Motor:** Asynchronous Python driver for MongoDB, ensuring that database I/O does not block the FastAPI event loop.
- **Local File System:** `uploads/videos/` and `uploads/thumbnails/` act as our Object Storage layer.

Security Architecture

1. **Stateless Authentication:** Upon login, the backend signs a JWT containing the user's ID and expiration time. The frontend stores this token in localStorage and attaches it as a Bearer token in the Authorization header of all subsequent protected API requests.
2. **Password Hashing:** Plain text passwords are never stored. bcrypt salts and hashes the password before insertion into MongoDB.

Data Flow (Video Streaming)

Streaming a video requires special handling to prevent loading a massive file entirely into the server's RAM.

- We utilize HTTP Range Requests (206 Partial Content).
- When the client's video player requests the video, it sends a Range header (e.g., bytes=0-10000).
- FastAPI reads only that specific byte range from the disk chunk and sends it back. This allows the user to scrub/seek through the video instantly.

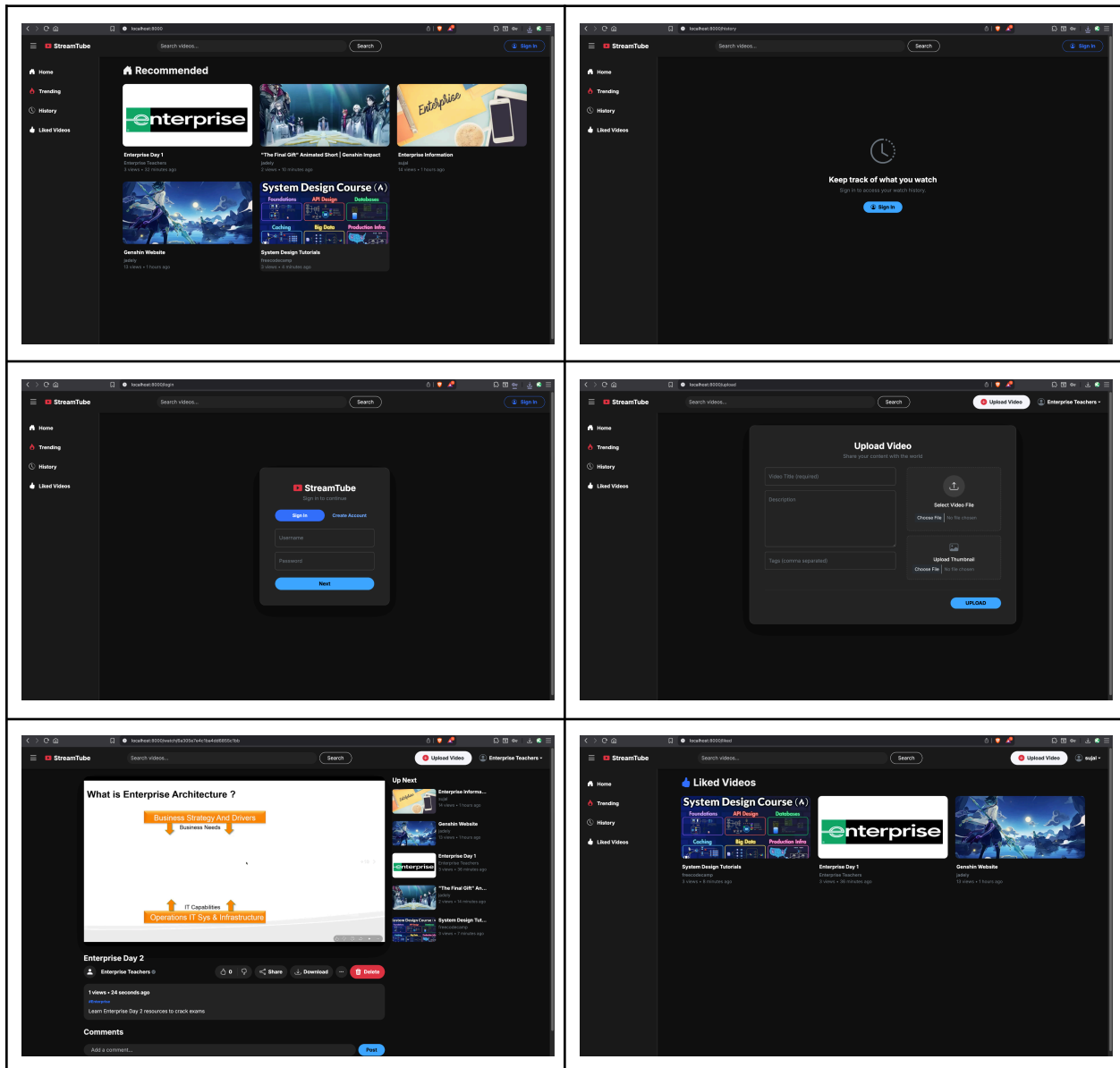
4. Module Descriptions

- **Auth:** Manages JWT creation and bcrypt hashing.
- **Videos:** Handles multipart uploads, thumbnail generation logic, and partial content delivery.
- **Social:** Allows toggling likes and adding comments.
- **Playlists:** Organizes video curations and tracks watch history.
- **Search:** Executes regex-based queries and tag matching in MongoDB.

5. Implementation Details

The backend is built in Python 3 using FastAPI. Authentication relies on OAuth2PasswordBearer. Video streaming is implemented via generators yielding byte chunks, explicitly setting `Accept-Ranges` and `Content-Range` headers to support native HTML5 `<video>` player seeking.

6. Screenshots



7. Future Scope

- **Object Storage:** Migrate from local file paths to AWS S3.
- **CDN Integration:** Use CloudFront to cache video chunks at edge nodes.
- **Transcoding:** Offload upload processing to Celery workers for multi-resolution encoding (HLS/DASH).
- **Search Engine:** Migrate text search from MongoDB to Elasticsearch for relevance scoring.